

Appendix N — The Five LLM Roles Reference

Memora never passes one shared LLM object through its pipeline. Both the original LangChain implementation (``app/``) and the later LangGraph rewrite (``app_workflow/``) construct a small, named roster of LLM instances at startup — each one a ``ChatOpenAI`` wrapper pointed at the same or a different backend, tuned with its own temperature and token budget for the one job it does. `Book_Index.md` names five roles across the project's history: ``llm``, ``judge_llm``, ``json_fix_llm``, ``llm_tool``, and ``merge_llm``. No single pipeline actually instantiates all five — the roster is a union across two codebases that solved the same merge-validation problem differently. This appendix is the reference for what each role does, which provider serves it in the code as it stands today, and how to swap that provider without touching a call site.

Definition — LLM role

A named ``ChatOpenAI`` (or ``ChatGroq``) instance built once in a pipeline's ``llm_setup.py`` module and imported by name everywhere it is needed, rather than passed around as a generic parameter. The name describes the job (``judge_llm`` judges, ``json_fix_llm`` repairs JSON), not the underlying model or provider — which is exactly what lets the provider change underneath a role without every caller being edited.

N.1 One project, two rosters

``app/llm_setup.py`` and ``app_workflow/services/llm_setup.py`` are separate files with separate module-level instances — importing ``llm`` from one does not give you the other's ``llm``. Table N.1 is the complete roster for both pipelines as the code stands today; a role marked — is not constructed in that pipeline at all.

Role	Purpose	<code>app/`</code> (LangChain)	<code>app_workflow/`</code> (LangGraph)
<code>llm</code>	General orchestration and drafting	✓ ChatOpenAI, custom endpoint	✓ ChatOpenAI, custom endpoint
<code>merge_llm</code>	NAC chunk-merge JSON production	✓ ChatOpenAI, custom endpoint	— folded into <code>llm</code>
<code>judge_llm</code>	Verdict / validation calls	✓ ChatOpenAI, custom endpoint	✓ ChatOpenAI, custom endpoint
<code>json_fix_llm</code>	Tier-3 LLM-based JSON repair	✓ ChatOpenAI, custom endpoint	✓ ChatOpenAI, custom endpoint
<code>llm_tool</code>	Tool-calling / query-variant generation	— not defined	✓ ChatOpenAI, custom endpoint

The asymmetry is architectural, not accidental. ``app/`` kept merge production as its own instance from the project's earliest design (ADR-018); ``app_workflow/``'s Neighbor-Aware Compression node (``nac.py``) never introduced a ``merge_llm`` at all — it imports only ``llm`` and ``judge_llm`` and routes merge production through ``llm``, validating the result with ``judge_llm``. ``llm_tool`` runs the opposite direction: it exists only in ``app_workflow/``, born from a later decision (ADR-060) to isolate tool-calling traffic onto its own instance, something ``app/``'s simpler agent loop never needed.

N.2 llm — general orchestration

`llm` is the workhorse role: draft generation, no-context answers, and any orchestration decision that is not a validation verdict or a JSON repair. Both pipelines build it the same way today, differing only in which config module supplies the defaults:

```
llm = ChatOpenAI(
    base_url=os.getenv("CUSTOM_API_BASE"),
    api_key=os.getenv("CUSTOM_API_KEY"),
    model=os.getenv("CUSTOM_API_MODEL_NAME", "llama-3.1-8b-instruct"),
    temperature=0.1,
    max_tokens=2048,
    max_retries=0,
)
```

`max\_retries=0` is deliberate and appears on every role in both files: LangChain's own retry loop is fully disabled because the project's FIFO call gate (Appendix M) owns all retry and rate-limit recovery end to end, and letting two retry mechanisms fight over the same 429 response was a real, closed bug rather than a hypothetical risk.

N.3 judge_llm — verdicts and validation

`judge\_llm` answers yes/no and structured-verdict questions: is this merge faithful to its source chunks, is this pair actually redundant, does this retrieval satisfy the query, is this final answer grounded. It runs at `temperature=0.0` in both pipelines — deterministic output matters more than fluency for a role whose entire job is a verdict a downstream `validate\_\*()` function will parse. `app\_workflow/nodes/nac.py` shows the pattern directly: it imports both `llm` and `judge\_llm` from the same module and passes `judge\_llm` specifically into `validate\_merge()`, keeping merge production and merge validation on two different instances even though `app\_workflow/` no longer has a dedicated `merge\_llm` to produce the merge in the first place.

```
# app_workflow/nodes/nac.py
from app_workflow.services.llm_setup import llm, judge_llm
# (chunk-merge production call via llm, elided -- covered in Appendix N.6)
check = validate_merge(pair["source_chunks"], pair["merged"], judge_llm,
                       config=config, switches=sw)
```

N.4 json_fix_llm — the last-resort JSON repair tier

`json_fix_llm` is only ever called after cheaper, non-LLM repair tiers (string preprocessing, then the `json_repair` library) have already failed to produce parseable, schema-valid output — it is Tier 3 of `fix_llm_output.py`'s repair pipeline, reserved for genuine format-translation failures a deterministic heuristic cannot fix (XML-shaped output, YAML, Python class syntax, in place of the requested JSON). Low temperature and a small token budget match that narrow job:

```
json_fix_llm = ChatOpenAI(
    base_url=os.getenv("CUSTOM_API_BASE"),
    api_key=os.getenv("CUSTOM_API_KEY"),
    model=os.getenv("CUSTOM_API_MODEL_NAME", "Qwen/Qwen2.5-Coder-3B-
Instruct"),
    temperature=0.0,
    max_tokens=1024,
    max_retries=0,
)
```

Common pitfall — A dead default hiding behind a live env var

The `"Qwen/Qwen2.5-Coder-3B-Instruct"` and `"Qwen/Qwen2.5-7B-Instruct"` default strings on `json_fix_llm` and `judge_llm` are leftovers from when those two roles were served by the Hugging Face Inference Providers router (ADR-054). Since ADR-062's full backend consolidation, `CUSTOM_API_MODEL_NAME` resolves `model=` identically for all four `app_workflow/` roles whenever it is set — so those Qwen defaults only matter on the rare path where `CUSTOM_API_MODEL_NAME` is unset entirely, and even then they name a model the current custom endpoint was never confirmed to serve. Decisions.md's own ADR-062 entry flags these as “dead HF-router-era values” in its Impact field — read a role's *actual* model from whatever `CUSTOM_API_MODEL_NAME` is set to in `.env`, not from the code's fallback string.

N.5 llm_tool — tool-calling, isolated on purpose

`llm\_tool` exists only in `app\_workflow/services/llm\_setup.py`, and its presence there is a fossil of a decision that has since been partly undone. ADR-060 (2026-07-03) split the roster in two: `llm`, `judge\_llm`, and `json\_fix\_llm` moved onto a new custom OpenAI-compatible endpoint, while the pipeline's original `ChatGroq` instance was kept alive under a new name, `llm\_tool`, reserved for tool-calling traffic — specifically `query\_variants.py`'s query-variant generation, which at the time depended on LangChain tool-call schemas for structured output and Groq was the one confirmed-working tool-calling backend. ADR-061, three days later, removed that dependency entirely — query-variant generation switched to a plain JSON array parsed through `fix\_llm\_output`, the same mechanism every other structured output in the project already used — which meant `llm\_tool`'s only remaining justification (tool-call support) had no live call site left. ADR-062 closed the loop: `llm\_tool` was migrated onto the same custom endpoint as the other three roles, and the `ChatGroq` construction block was commented out in place rather than deleted, preserving a fast rollback path if the custom endpoint ever needs to be abandoned again.

```
# app_workflow/services/llm_setup.py
# llm_tool = ChatGroq(
#     api_key=os.getenv("GROQ_API_KEY"),
#     model_name=os.getenv("GEN_MODEL_NAME", "llama-3.1-8b-instant"),
#     temperature=0.1,
#     max_tokens=2048,
#     max_retries=0,
# )

llm_tool = ChatOpenAI(
    base_url=os.getenv("CUSTOM_API_BASE"),
    api_key=os.getenv("CUSTOM_API_KEY"),
    model=os.getenv("CUSTOM_API_MODEL_NAME", "llama-3.1-8b-instruct"),
    temperature=0.1,
    max_tokens=2048,
    max_retries=0,
)
```

`llm\_tool` and `llm` are, as of today's code, configured identically — same endpoint, same temperature, same token budget. The distinction that once separated them (tool-call support) no longer does any real work, but the two names are kept separate rather than collapsed back into one, so that the one call site that still imports `llm\_tool` (`query\_variants.py`) can be redirected to a different backend later without touching every other node that imports `llm`.

N.6 merge_llm — a role app_workflow/ chose not to keep

`merge\_llm` is `app/`'s answer to Neighbor-Aware Compression's chunk-merge step, and it is the one role that never made the jump to `app\_workflow/`. ADR-018 is explicit about why the classic pipeline kept it separate from `judge\_llm` in the first place: the two roles were expected to diverge eventually — `judge\_llm` upgraded to a stronger model for faithfulness evaluation while `llm` stayed fast, and `merge\_llm` tuned toward deterministic JSON output at its own temperature — and instantiating three separate objects up front costs nothing while leaving that hybrid-model door open.

```
# app/llm_setup.py
merge_llm = ChatOpenAI(
    base_url=os.getenv("CUSTOM_API_BASE"),
    api_key=os.getenv("CUSTOM_API_KEY"),
    model=os.getenv("CUSTOM_API_MODEL_NAME", "llama-3.1-8b-instruct"),
    temperature=0.0,
    max_tokens=2048,
    max_retries=0,
)

# app/tools.py — merge_llm is optional; judge_llm is the fallback
chosen_llm = merge_llm if merge_llm is not None else judge_llm
```

`app\_workflow/`'s `nac.py` never grew an equivalent instance — it produces a merge with `llm` and validates it with `judge\_llm`, folding what `app/` treats as a third, independently-tunable role into the two roles the LangGraph pipeline already has. Table N.1 records this as “folded into llm” rather than “missing” for exactly this reason: it is a design choice made once the pipeline was rewritten, not an oversight — read Architecture.md's own noted improvement direction (moving `merge\_llm`/`judge\_llm` to a stronger model than the fast `instant` variant) if you are deciding whether your own fork should keep the two roles separate or reunify them.

N.7 How to swap: the env-var contract every role shares

Every current role in both files is built from the same three environment variables — `CUSTOM\_API\_BASE`, `CUSTOM\_API\_KEY`, `CUSTOM\_API\_MODEL\_NAME` — which is what makes swapping the underlying model or provider a `.env` edit rather than a code change. Nothing about `ChatOpenAI` requires the endpoint to actually be OpenAI's; any OpenAI-compatible chat-completions server (a local TGI/vLLM instance, Groq, the Hugging Face Inference Providers router) satisfies the same three parameters. This provider-agnostic wrapper choice predates the current roster (it traces back to ADR-006/ADR-014) and is exactly what let the roster move providers four times — Groq only, then Groq plus the HF router, then a three-role custom-endpoint split with Groq kept for tools, then full consolidation — in the space of about two weeks (ADR-054 → ADR-060 → ADR-061 → ADR-062) without a single node module importing anything differently.

- To move a role to a different OpenAI-compatible server: change `CUSTOM\_API\_BASE`/`CUSTOM\_API\_KEY`/`CUSTOM\_API\_MODEL\_NAME` in `.env` — every role built with the ADR-062 pattern picks it up on next startup.
- To move a role back to native Groq: uncomment the matching `ChatGroq` block already preserved in place in `llm\_setup.py` and comment out the `ChatOpenAI` block — both files keep the prior provider's construction code rather than deleting it, specifically for this rollback path.
- To give one role a genuinely different model than the rest: set that role's `model=` argument to a literal string instead of `os.getenv("CUSTOM\_API\_MODEL\_NAME", ...)`; the same way `judge\_llm` and `json\_fix\_llm` differ from `llm` in temperature and token budget today.

Common pitfall — CUSTOM_API_BASE pointed at the wrong URL shape

`ChatOpenAI` expects `base_url` to be the server root (e.g. `http://localhost:8080/v1`), not a full request path. `Decisions.md` records a helper, `_normalize_openai_base_url()`, that strips a trailing `/chat/completions` (and `/completions`, `/responses`) from `CUSTOM_API_BASE` before it reaches `ChatOpenAI` — added specifically because a local server's own displayed “endpoint URL” is often the full completions path, and pasting that in directly causes every request to double-append the suffix. If a role starts failing immediately after a `.env` change with a 404 that mentions a doubled path segment, this is the first thing to check.